UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3092 – Deep Learning



Laboratorio 9

Series de tiempo y forecasting

Diego Rosales - 23258
Juan Francisco Martínez - 23617
Link de repositorio: https://github.com/diegoanro22/DL-LAB09

GUATEMALA, 1 de noviembre del 2026

1. Datos, preparación y protocolo

Se pronostican 24 horas de consumo de 150 clientes portugueses (ElectricityLoadDiagrams20112014), tomados con semilla 42 de los 315 que tienen datos completos desde 2012. Los registros de 15 minutos en kW se suman y multiplican por 0.25 para obtener kWh por hora. En el cambio de horario de marzo la hora vacía se interpola y en octubre la hora duplicada se divide entre dos. Las escalas van de 8.3 a 48,817 kWh/h, así que cada serie se normaliza con su media y desviación de entrenamiento. En Navidad el consumo cae entre 29 y 33 %. Las covariables conocidas a futuro son hora, día y mes en seno y coseno, más un indicador de feriado portugués. Entrenamiento llega al 30 de junio de 2014, validación cubre julio a septiembre y prueba octubre a diciembre; cada serie tiene 26,304 horas (3,945,600 en el panel) y el Dataset de PyTorch produce, con L = 168, H = 24 y stride de 1 h, 3,254,550 ventanas de entrenamiento y 327,750 de validación y de prueba.

Todos los modelos pasan por la misma función backtest(): 92 orígenes a medianoche en prueba, historia cortada en origen − 1 h y pesos entrenados una vez con datos hasta el 30 de septiembre. Con el seasonal naive, backtest() coincide exactamente con StatsForecast.cross_validation. Se auditaron cuatro fugas: multiplicar por 100 los datos de validación y prueba no cambia los estadísticos del escalado; la entrada termina en origen − 1 h y el calendario del horizonte coincide hora por hora con las fechas objetivo; un split aleatorio haría que el 100 % de las ventanas de validación compartieran horas con entrenamiento, por eso los bloques son cronológicos; y el entrenamiento lanza un error si la validación del early stopping llega a la prueba.

2. Investigación

nn.LSTM devuelve la secuencia de estados y el par final (h, c), y nn.GRU solo h. Con ese estado se puede predecir un paso y realimentarlo, emitir 24 pasos con una capa densa o pasarlo a un decoder; en seq2seq, teacher forcing da al decoder la etiqueta real durante el entrenamiento y su propia predicción en inferencia. nn.Conv1d espera (batch, canales, tiempo) y con convoluciones causales dilatadas el campo receptivo es $R = 1 + \Sigma(k - 1)d$ (127 horas con kernel 3 y dilataciones 1 a 32). TransformerEncoderLayer no conoce el orden de los tokens y necesita codificación posicional. En Nixtla, cross_validation con n_windows = 92, step_size = 24 y refit = False reproduce el protocolo, con origen = cutoff + 1 h. Chronos-Bolt expone predict_quantiles, que devuelve (series, horizonte, cuantiles). La pinball del cuantil τ vale $\tau \cdot u$ si $u = y - q > 0$ y $(\tau - 1) \cdot u$ si no. MASE pesa igual a cada cliente y WAPE queda dominado por los grandes; el MAPE se dispara con clientes de consumo cercano a cero, por eso se excluyen las horas con $|y| \leq 0.001$; RevIN normaliza cada ventana y revierte la salida.

3. Iteraciones de hiperparámetros

La selección se hizo solo con validación y cada iteración registró configuración, pérdidas, MAE y MASE por epoch, parámetros, tiempos y memoria de GPU (outputs/runs). Todos los modelos se entrenaron en una RTX 5070 Laptop con presupuesto comparable: 1,500 pasos de 1,024 ventanas en NeuralForecast y hasta 15 epochs de 1,000 batches de 512 en las redes propias. Una primera versión de las recurrentes, N-HiTS, TCN y DeepAR entrenada en CPU con unas 100 veces menos ventanas perdía contra el seasonal naive y se descartó.

Modelo	L = 48	L = 168	L = 336	3 meses	12 meses	30 meses
Seasonal naive 24	0.985	0.985	0.985	0.985	0.985	0.985
LightGBM	0.952	0.857	0.857	0.924	0.859	0.857
DLinear	0.965	0.880	0.848	0.885	0.882	0.880
PatchTST	0.944	0.857	0.834	0.894	0.880	0.857
LSTM directa	1.025	0.988	1.019	1.550	1.005	0.988

Tabla 1. MASE de validación al variar la ventana L y los meses de historia

Una ventana más larga ayudó a PatchTST y DLinear (0.944 a 0.834 en PatchTST); en LightGBM solo importa si cabe el rezago de 168 h. RevIN apenas cambió a los lineales y mejoró poco al MLP (0.830 a 0.820). Un MLP mayor (1024 × 3) bajó su pérdida de entrenamiento pero empeoró a 0.863, y en la LSTM autorregresiva la pérdida de entrenamiento siguió bajando tras el epoch 11 mientras el MASE de validación subía de 1.145 a 1.20: menor pérdida de entrenamiento no implica mejor validación.

4. Estrategias multi-paso y comparación

Con la misma LSTM, la estrategia autorregresiva obtuvo MASE de prueba de 1.159, la multi-salida 0.983 y el encoder–decoder 0.947. En la Figura 1, el MAE autorregresivo pasa de 54.7 kWh en h = 1 a 93.1 en h = 24, mientras las directas terminan entre 60 y 70: es la acumulación de errores al realimentar predicciones. En el resto de familias el error sigue la forma del día y es mayor entre el mediodía y la tarde.

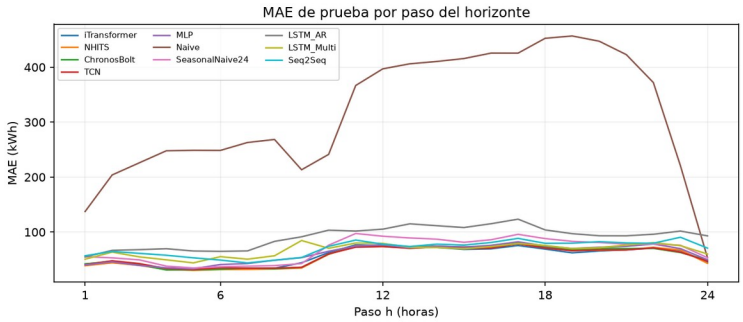


Figura 1. MAE de prueba por paso del horizonte

La Tabla 2 compara todos los modelos en prueba y la Tabla 3 los dos probabilísticos. El MASE con H = 168 no aplica porque solo se entrenó H = 24; LightGBM cuenta nodos en lugar de pesos y MSTL se ajusta en cada origen, por eso su tiempo es de inferencia.

Modelo	Familia	Estrategia	Cov.	Parám.	Trains	Infer. s	MAE	RMSE	WAPE	MASE	MASE 22–28 dic
iTransformer	Atención	Directa	No	1.63M	76	16	56.6	504	0.072	0.809	1.386
N-HiTS	Residual	Directa	No	2.82M	36	12	56.3	494	0.072	0.816	1.524
Chronos-Bolt	Fundac.	Zero-shot	No	47.72 M	0	85	57.2	499	0.073	0.818	1.486
TCN	Convol.	Directa	Sí	271k	270	12	56.6	495	0.072	0.826	1.519
MLP	Lineal	Directa	Sí	1.05M	165	14	60.9	526	0.078	0.852	1.506
LightGBM	Árboles	Recursiva	Sí	305k	48	25	59.5	526	0.076	0.867	1.578
PatchTST	Atención	Directa	No	467k	148	17	60.5	526	0.077	0.882	1.660
NLinear	Lineal	Directa	No	4k	172	18	65.6	610	0.084	0.913	1.827
DLinear	Lineal	Directa	No	8k	77	14	65.2	603	0.083	0.914	1.825
MSTL	Estadíst.	Directa	No	local	0	435	65.8	571	0.084	0.917	2.066
Seq2Seq	Recurr.	Seq2seq	Sí	141k	449	11	70.3	583	0.090	0.947	1.500
LSTM directa	Recurr.	Directa	Sí	112k	375	9	67.3	508	0.086	0.983	1.634
SNaive 24	Baseline	Copia	No	0	0	13	67.9	546	0.087	1.055	2.313
SNaive 168	Baseline	Copia	No	0	0	14	80.0	888	0.102	1.106	1.661
LSTM AR	Recurr.	Recursiva	Sí	19k	99	9	91.3	800	0.117	1.159	1.792
DeepAR	Probab.	Recursiva	Sí	71k	178	16	102.8	759	0.132	1.363	2.058
Naive	Baseline	Copia	No	0	0	13	315.4	2370	0.404	4.298	4.274

Tabla 2. Resultados en prueba ordenados por MASE (H = 24, 92 orígenes, L = 168 salvo baselines; parámetros entrenables iguales a los totales salvo Chronos, con 0; RTX 5070, Chronos en CPU)

Modelo	Pinball 0.1	Pinball 0.5	Pinball 0.9	Pinball media	Cobertura P10–P90
Chronos-Bolt	14.3	28.6	14.0	19.0	81.2 %
DeepAR	25.4	52.5	24.3	34.1	67.4 %

Tabla 3. Pérdida pinball y cobertura del intervalo P10–P90 en prueba

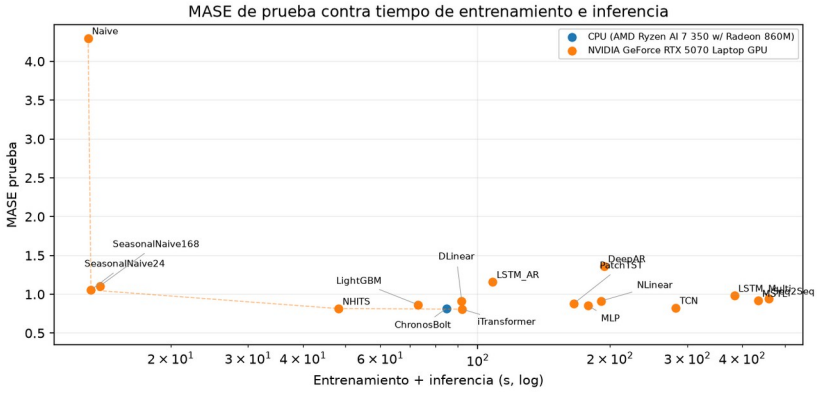


Figura 2. MASE de prueba contra tiempo de entrenamiento e inferencia (escala log)

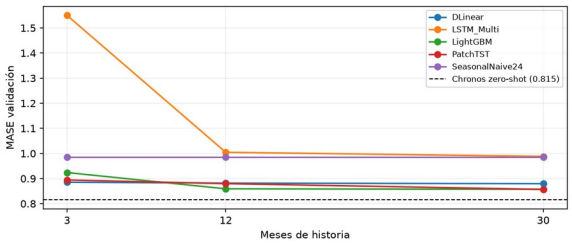


Figura 3. MASE de validación contra meses de historia, con Chronos como referencia

5. Discusión y conclusiones

iTransformer ganó con MASE 0.809, 23 % por debajo del seasonal naive (1.055), seguido por N-HiTS (0.816), Chronos (0.818) y TCN (0.826). N-HiTS entrena en 36 s y con tres semillas da 0.814, 0.801 y 0.808 (media 0.808, desviación 0.006), igual a la distancia entre los cuatro primeros, así que en producción es la opción más razonable. El hallazgo de DLinear se replica a medias: DLinear y NLinear (0.914, con 8 mil y 4 mil parámetros) superan a MSTL y al seasonal naive, y con L = 336 alcanzan a PatchTST en validación, pero los modelos con atención les sacan entre 18 y 26 % en h = 12 a 18. Con 3 meses DLinear le gana a LightGBM y PatchTST y la LSTM sube a 1.550; con 30 meses PatchTST solo empata a LightGBM (0.857), así que el deep learning supera a los estadísticos desde 3 meses pero nunca a LightGBM en validación.

El modelo global de la LSTM obtuvo MASE de validación de 1.040 en 10 clientes, contra 1.582 de diez modelos locales; convino el global, aunque los locales tuvieron menos presupuesto y no se guardó el detalle por cliente para ver quién gana más. Chronos sin entrenar quedó a 0.009 del ganador, obtuvo 0.815 en validación y su intervalo P10–P90 cubrió 81 % (DeepAR, 67 %); sirve para clientes nuevos o sin tiempo de entrenar, aunque estos datos pudieron estar en su preentrenamiento. En la semana del 22 al 28 de diciembre todos empeoran (iTransformer sube a 1.386), con o sin covariable de feriado, porque cambia el régimen de toda la semana; el seasonal naive de 168 h se degrada menos que el de 24 h. Reentrenar cada mes mejoró poco a N-HiTS (0.816 a 0.810) y no a LightGBM (0.867 a 0.868). En conclusión, el presupuesto de entrenamiento pesó más que la arquitectura.

6. Referencias

Ansari, A. F., et al. (2024). Chronos: Learning the language of time series. https://arxiv.org/abs/2403.07815

Zeng, A., et al. (2023). Are transformers effective for time series forecasting? https://arxiv.org/abs/2205.13504